Tell Me Why: Language Models Help Explain the Rationale Behind Internet Protocol Design

Jie Bian [1] Michael Welzl [1] Andrey Kutuzov [1] Nikolay Arefyev [1]

[1]Department of Informatics, University of Oslo

Summary

Request for Comments (RFCs) serve as guidebooks for the implementation of Internet protocols or network mechanisms. They reveal how these protocols and mechanisms work, but the underlying reasons for their operation are not always available in RFCs. Using natural language processing techniques, we present an attempt to automate the discovery of these reasons from mailing list archives.

The Story of Winston Webber

Winston Webber is a programmer who works on a new client for WebRTC (a set of standards that enable peer-to-peer media communication between browsers [2]).

Winston asks ChatGPT for Help...

Winston tries the following query: *"RFC 7657 says, in section 6, the applications: 'Should use a single DSCP for RTCP packets, which should be a DSCP used for RTP packets that are or would be sent by that SSRC'. Why?"*

ChatGPT produces the following three reasons (shortened):

1. '*Congruent Treatment:* RTP and RTCP packets are closely related, with RTP carrying media streams and RTCP providing control and feedback. Assigning them the same DSCP ensures consistent treatment.'
2. '*QoS Differentiation:* Network devices can prioritize traffic based on DSCP. By using the same DSCP for RTP and RTCP packets, consistent QoS policies can be applied, maintaining relative ordering, delay, and loss characteristics.'
3. '*Flow Identification:* Assigning the same DSCP simplifies flow identification, allowing network devices to recognize the relationship between RTP and RTCP packets. This enables appropriate traffic management and QoS enforcement.'

What are their intended meanings?

1. It just states that using the same DSCP yields consistent treatment of RTP and RTCP packets. RFC 7657 contains text explaining why: such consistent treatment can improve the quality of Round-Trip Time (RTT) calculations based on RTCP feedback.
2. same as above
3. It is not unreasonable to assume that flows could be more easily identified when the same DSCP value is applied for RTP and RTCP packets. On the other hand, the DSCP would be quite an unreliable input for flow identification, as it is prone to change in transit. Was this *really* a reason to specify this guideline?

...But It Cannot Be Trusted.: Winston has an inkling that ChatGPT may be hallucinating. In the wake of the Snowden affair, the IETF has called out pervasive monitoring as a "technical attack that should be mitigated in protocol design, where possible" (RFC7258); the QUIC protocol, which encrypts not only data but also protocol headers (RFC9000), is a testament to this decision. Winston is right: no discussion of flow identification has taken place in the mailing list or at meetings at all (we confirmed this by checking the emails and meeting minutes), and this reason is, therefore, a ChatGPT invention.

A Feasible Tool

He needs a tool that makes it easier to find the right email conversation. In this paper, we attempt to answer the question: *would the construction of such a tool be feasible?* Specifically, for "feasibility", we require:

- The **positive class precision**, i.e., the probability of finding the correct discussion when clicking a paragraph, is approximately 70%.
- **Positive class recall** is such that, **for every RFC, at least a handful of paragraphs** are found.

Dataset

We leverage the known relationship between text changes and the discussions leading to these changes in the Working Group (WG) GitHub repositories of the Internet Engineering Task Force (IETF) to obtain labeled training data.

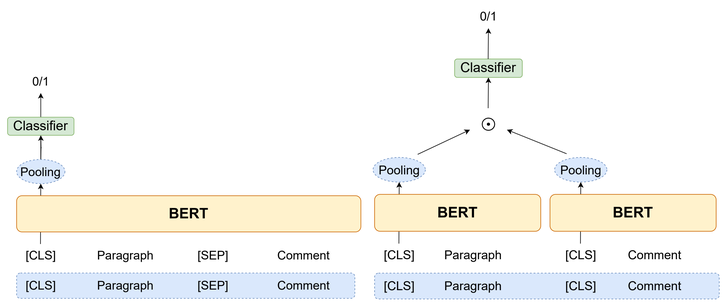
We take all the '*PR paragraph:commentary*' relations we get from Github as positive instances and generate the negative instances via random sampling within the same GitHub repository. We assign subsets such that 20% of the data are used as validation set and the remaining 80% as training.

Table 1. Training and validation datasets statistics.

Set	Subset	Repos.	I-Ds	Files	PosTotal
Valid	1	39	46	4,106	18,363
Train	2	12	24	5,648	23,158
	3	24	31	2,868	12,360
	4	40	50	4,238	18,161
	5	51	58	5,292	17,520
Email					
Test		RFC 7657	1	7	167
		RFC 8335	1	8	72

Model Design

We train the models for a binary classification task using two different methods. The first method, referred to as Model Type A, utilizes a joint encoder (as shown in the left figure). The second method, Model Type B, employs a bi-encoder (illustrated in the right figure).



Evaluation

Table 2 shows the model performance with the default threshold. Since the precision reported in this table is below our desired value of 70%, we need to increase it.

Finally, we check the models on a real-scenario dataset from mailing lists. We use the test dataset that we collect from the email archive and manually annotate, and results are reported in Table 3.

Table 2. Performance with validation set

Model	Class	Precision	Recall	MF1
A	neg	0.66	0.64	0.65
	pos	0.65	0.67	0.66
A+	neg	0.63	0.69	0.66
	pos	0.66	0.60	0.63
B	neg	0.59	0.64	0.61
	pos	0.61	0.56	0.58
B+	neg	0.58	0.66	0.62
	pos	0.61	0.53	0.57

Table 3. Performance with the test dataset with different thresholds

Model	RFC	Class	Prec/Prec+	Rec/Rec+	MF1/MF1+
A	7657	neg	0.71/0.68	0.64/0.74	0.67/0.71
		pos	0.70/0.74	0.76/0.68	0.73/0.71
	8335	neg	0.66/0.63	0.59/0.70	0.62/0.66
		pos	0.67/0.71	0.74/0.64	0.70/0.67
A+	7657	neg	0.64/0.58	0.87/0.96	0.74/0.73
		pos	0.82/0.91	0.55/0.37	0.66/0.53
	8335	neg	0.62/0.54	0.71/0.78	0.66/0.64
		pos	0.71/0.68	0.61/0.42	0.66/0.52
B	7657	neg	0.54/0.51	0.89/0.97	0.67/0.67
		pos	0.76/0.83	0.32/0.14	0.45/0.24
	8335	neg	0.47/0.47	0.89/0.95	0. 61/0.62
		pos	0.53/0.50	0.11/0.04	0.18/0.08
B+	7657	neg	0.65/0.56	0.69/0.91	0.66/0.69
		pos	0.69/0.80	0.65/0.34	0.67/0.47
	8335	neg	0.48/0.48	0.83/0.94	0.60/0.63
		pos	0.58/0.64	0.21/0.10	0.31/0.17

Evaluation

We present the pairs correctly identified as positive with Models A and B (threshold 0.7) in Table 4 based on the validation set. It is noteworthy that, despite the high threshold value, Model A still identifies positive instances for all documents, whereas Model B does not make predictions for several drafts.

Table 4. Prediction of positive instances per draft on validation set

I-Ds (draft- or RFC)	TP(A)	TP(B)	TP+FN	I-Ds (draft- or RFC)	TP(A)	TP(B)	TP+FN
add-requirements	50	0	99	ietf-privacypass-auth-scheme	102	44	181
ietf-ace-oauth-authz	81	33	313	ietf-privacypass-protocol	248	157	529
ietf-add-split-horizon-authority	37	57	78	ietf-quic-datagram	72	17	120
ietf-anima-autonomic-control-plane	448	25	781	ietf-rats-uccs	12	0	17
ietf-ccamp-optical-impairment-topo...	24	4	71	ietf-sacm-requirements	2	5	53
ietf-ccamp-rfc9093-bis	5	2	5	ietf-sedate-datetime-extended	54	47	101
ietf-core-senml	143	103	282	ietf-suit-architecture	308	219	489
ietf-emu-aka-pfs	22	0	28	ietf-suit-firmware-encryption	62	30	151
ietf-gnap-resource-servers	84	0	173	ietf-tls-dtls-connection-id	132	9	283
ietf-homenet-front-end-naming-dele...	126	33	692	ietf-tls-iana-registry-updates	138	31	235
ietf-httpapi-ratelimit-headers	316	257	507	irtf-nwcrg-network-coding-satellites	119	27	144
ietf-httpapi-rfc7807bis	53	0	102	RFC 8152	705	555	2448
ietf-ice-pac	21	0	39	RFC 8555	1005	452	1970
ietf-lamps-documentsigning-eku	57	34	139	RFC 8922	353	132	656
ietf-mls-protocol	2556	1786	4084	RFC 9243	6	17	49
ietf-ppm-dap	602	294	1089	RFC 9265	309	122	569
ietf-privacypass-architecture	516	52	809	RFC 9360	33	17	131

Conclusion

Understanding the design rationale of network protocols is valuable for teaching, protocol design, and decision-making for implementation, as seen in Winston Webber's case.

We find that our model is able to generalize and actually discover emails that have led to formulations in RFCs. This is a first step towards facilitating a deeper understanding of these often complex documents, which can be helpful to developers, protocol designers, and educators alike.

This work opens up opportunities for various follow-up activities.

The End of Winston Webber's Story

Our model A yields several high-valued (e.g. over 0.8) predictions for emails coming from the same thread that is relevant to the quoted paragraph from Section 6 of RFC 7657. These emails clarify the views of the discussion participants:

- They agree that feedback messages are important.
- RTT estimation is their primary (if not only) reason to recommend the same DSCP value for RTP and RTCP.
- They discuss the fact that different media from the same SSRCs can carry different DSCP values. This automatically renders the specific DSCP choice for RTCP-per-SSRC an open question.
- The discussion culminates in: "RTCP for each SSRC should use the DSCP with the lowest drop preference of those used for the media sent by that SSRC". This is semantically equivalent to the final text in the RFC's Section 5.4.

Winston now understands: the first rule is to use the same DSCP as for RTP, for the sake of RTT estimation. When this is not possible (because "the same" becomes ambiguous), the second rule of allowing a low drop precedence applies. Also, except for RTT estimation, it seems that the contributors of RFC 7657 favor DSCP choices that might increase the probability of getting RTCP feedback to the sender.

Our model made Winston happy!

References

[1] Chris Alberti, Jeffrey Ling, Michael Collins, and David Reitter. Fusion of detected objects in text for visual question answering. In *Proc. of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[2] Harald T. Alvestrand. Overview: Real-Time Protocols for Browser-Based Applications. RFC 8825, January 2021.

[3] Sebastian Benthall. Testing generative models of online collaboration with bigbang. In *Proc. of the 14th Python in Science Conference*, pages 182–189, 2015.

[4] Manoj Bhat, Klym Shumaiev, Uwe Hohenstein, Andreas Biesdorf, and Florian Matthes. The evolution of architectural decision making as a key focus area of software architecture research: A semi-systematic literature study. In *2020 IEEE ICSA*, pages 69–80. IEEE, 2020.